Case Study: COVID-19

COVID-19 Case Study:

Overview: *The COVID-19 pandemic, caused by the SARS-CoV-2 virus, emerged in late 2019 and rapidly spread globally, leading to significant health, economic, and social impacts. This unprecedented health crisis highlighted the crucial role of data analysis in managing such pandemics. By meticulously tracking and analyzing data on confirmed cases, recoveries, and deaths, policymakers and health professionals can make informed decisions to control the spread of the virus and allocate resources effectively.*

Dataset Details:
Dataset *(Open link in a new tab and download the dataset.)*
This case study utilizes three key datasets, each providing daily updates on different aspects of the pandemic for various countries and regions:

- **Confirmed Cases Dataset:** *Contains the cumulative number of confirmed COVID-19 cases per day for each country and region. The data spans from January 22, 2020, to May 29, 2021, with over 276 geographic entries.*
- **Deaths Dataset:** *Records the cumulative number of deaths attributed to COVID-19, structured similarly to the confirmed cases dataset. This provides crucial information for assessing the lethality and outbreak severity in different areas.*
- **Recovered Cases Dataset:** *Includes data on the cumulative number of individuals who have recovered from COVID-19, which is vital for understanding the disease's progression and the effectiveness of treatment protocols.*

Each dataset includes columns for Province/State, Country/Region, geographic coordinates (Lat, Long), and a series of dates representing daily cumulative totals.

Objective of the Case Study:
The primary objectives of this case study are:

- **Practical Application of Python:** *To demonstrate and enhance practical skills in using Python for data analysis, focusing on data manipulation, cleaning, and visualization.*
- **Insightful Data Analysis:** *To provide analytical insights into the dynamics of COVID-19's spread, recovery, and mortality rates across different regions and over time.*
- **Skill Development:** *To equip students with the knowledge to handle real-world data using Python libraries such as Pandas for data manipulation, Matplotlib for data visualization, and Numpy for numerical data processing.*

Analysis Questions:

1. ***Data Loading:*** *How do you load the COVID-19 datasets for confirmed cases, deaths, and recoveries into Python using Pandas?*

2. ***Data Exploration***

   1. *After loading the datasets, what is the structure of each dataset in terms of rows, columns, and data types?* `.info()`
   2. *Generate plots of confirmed cases over time for top countries.*
   3. *Generate plots of confirmed cases over time for China.*

3. ***Handling Missing Data:*** *Identify these missing values and replace them using a suitable imputation method such as forward filling for time-series data.*

4. ***Data Cleaning and Preparation:*** *Replace blank values in province column with "All Provinces"*

5. ***Independent Dataset Analysis***

   1. *Analyze the peak number of daily new cases in Germany, France, and Italy. Which country experienced the highest single-day surge, and when did it occur?*
   2. *Compare the recovery rates (recoveries/confirmed cases) between Canada and Australia as of December 31, 2020. Which country showed better management of the pandemic according to this metric?*
   3. *What is the distribution of death rates (deaths/confirmed cases) among provinces in Canada? Identify the province with the highest and lowest death rate as of the latest data point.*

6. ***Data Transformation***

   1. *Transform the 'deaths' dataset from wide format (where each column represents a date) to long format where each row represents a single date and columns are now country names, ensuring that the date column is in datetime format. How would this transformation be executed?* `.melt()`
   2. *What is the total number of deaths reported per country up to the current date?*
   3. *What are the top 5 countries with the highest average daily deaths?*
   4. *How have the total deaths evolved over time in the United States?*

7. *Data Merging*

1. *How would you merge the transformed datasets of confirmed cases, deaths, and recoveries on the 'Country/Region' and 'Date' columns to create a comprehensive view of the pandemic's impact?*
2. *Analyze the monthly sum of confirmed cases, deaths, and recoveries for countries to understand the progression of the pandemic.[From the merged dataset]*
3. *Redo the analysis in last question for United States, Italy and Brazil.*

8. *Combined Data Analysis*

1. *For the combined dataset, identify the three countries with the highest average death rates (deaths/confirmed cases) throughout 2020. What might this indicate about the pandemic's impact in these countries?*
2. *Using the merged dataset, compare the total number of recoveries to the total number of deaths in South Africa. What can this tell us about the outcomes of COVID-19 cases in the country?*
3. *Analyze the ratio of recoveries to confirmed cases for the United States on a monthly basis from March 2020 to May 2021. Which month experienced the highest recovery ratio, and what could be the potential reasons?*

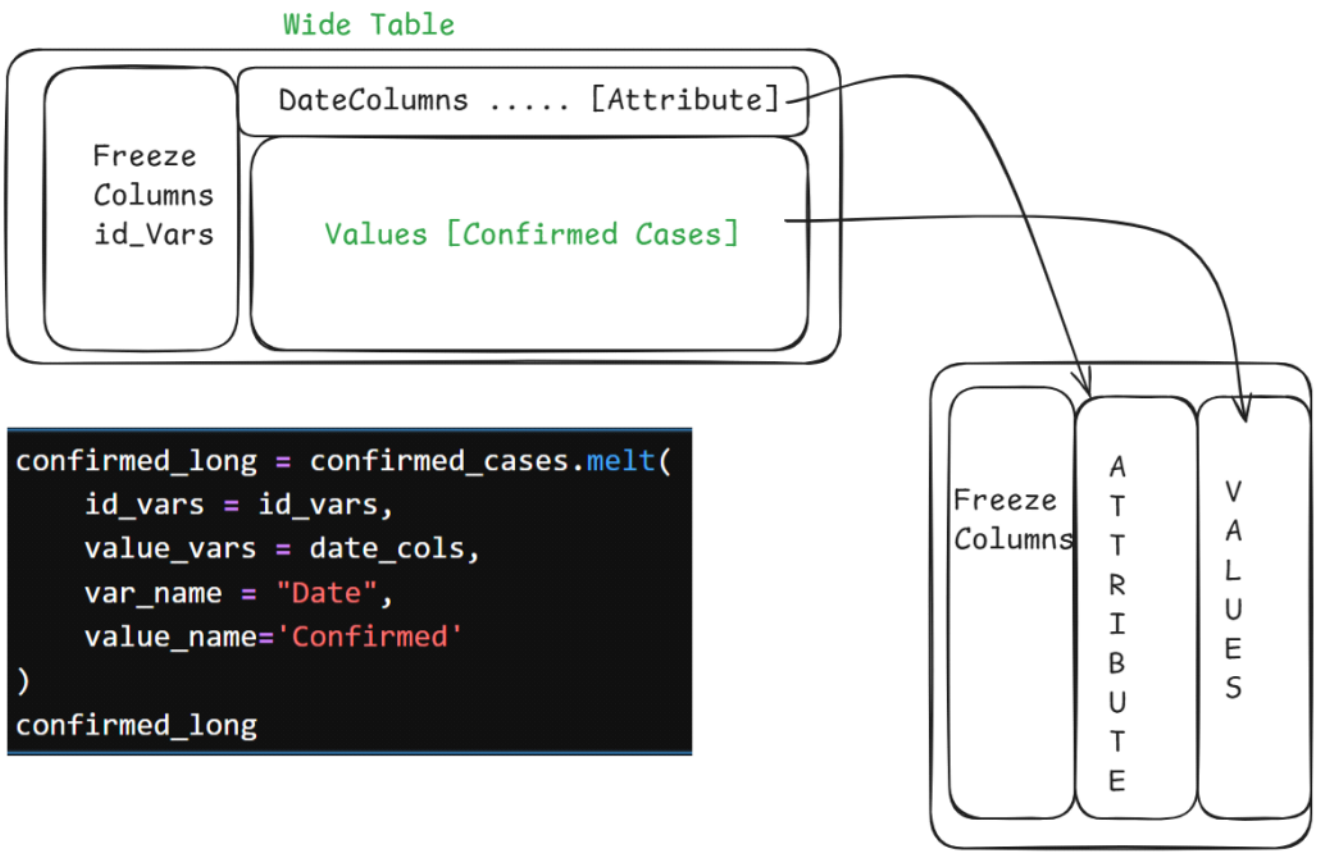
Attribute

	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	...	5/20/21
0	NaN	Afghanistan	33.93911	67.709953	0	0	0	0	0	0	...	64575
1	NaN	Albania	41.15330	20.168300	0	0	0	0	0	0	...	132118
2	NaN	Algeria	28.03390	1.659600	0	0	0	0	0	0	...	126156
3	NaN	Andorra	42.50630	1.521800	0	0	0	0	0	0	...	13569
4	NaN	Angola	-11.20270	17.873900	0	0	0	0	0	0	...	31661

Values

5 rows x 498 columns

Freeze Columns [id_vars]




```python
confirmed_long = confirmed_cases.melt(
    id_vars = id_vars,
    value_vars = date_cols,
    var_name = "Date",
    value_name='Confirmed'
)
confirmed_long
```

Passing `errors='coerce'` will force an out-of-bounds date to `NaT`, in addition to forcing non-dates (or non-parseable dates) to `NaT`.

```python
pd.to_datetime("invalid for Ymd", format="%Y%m%d", errors="coerce
```

```python
import pandas as pd
confirmed_cases = pd.read_csv('covid_19_confirmed.csv')
confirmed_cases.head()
```

	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	...	5/20/21
0	NaN	Afghanistan	33.93911	67.709953	0	0	0	0	0	0	...	64575
1	NaN	Albania	41.15330	20.168300	0	0	0	0	0	0	...	132118
2	NaN	Algeria	28.03390	1.659600	0	0	0	0	0	0	...	126156
3	NaN	Andorra	42.50630	1.521800	0	0	0	0	0	0	...	13569
4	NaN	Angola	-11.20270	17.873900	0	0	0	0	0	0	...	31661

5 rows × 498 columns

```python
confirmed_cases.shape
```

```
(276, 498)
```

```python
276 * 494
```

```
136344
```

```
deaths = pd.read_csv('covid_19_deaths.csv')
deaths.head()
```

	Column1	Column2	Column3	Column4	Column5	Column6	Column7	Column8	Column9	Column10	...	Colum
0	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	...	5/
1	NaN	Afghanistan	33.93911	67.709953	0	0	0	0	0	0	...	
2	NaN	Albania	41.1533	20.1683	0	0	0	0	0	0	...	
3	NaN	Algeria	28.0339	1.6596	0	0	0	0	0	0	...	
4	NaN	Andorra	42.5063	1.5218	0	0	0	0	0	0	...	

5 rows × 498 columns

```
# Promoting Headers
deaths = pd.read_csv('covid_19_deaths.csv' , header=1)
deaths.head()
```

	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	...	5/20/21
0	NaN	Afghanistan	33.93911	67.709953	0	0	0	0	0	0	...	2772
1	NaN	Albania	41.15330	20.168300	0	0	0	0	0	0	...	2440
2	NaN	Algeria	28.03390	1.659600	0	0	0	0	0	0	...	3401
3	NaN	Andorra	42.50630	1.521800	0	0	0	0	0	0	...	127
4	NaN	Angola	-11.20270	17.873900	0	0	0	0	0	0	...	704

5 rows × 498 columns

```
deaths.shape
```

```
(276, 498)
```

```
recovered_cases = pd.read_csv('covid_19_recovered.csv')
recovered_cases.head()
```

	Column1	Column2	Column3	Column4	Column5	Column6	Column7	Column8	Column9	Column10	...	Colum
0	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	...	5/
1	NaN	Afghanistan	33.93911	67.709953	0	0	0	0	0	0	...	
2	NaN	Albania	41.1533	20.1683	0	0	0	0	0	0	...	1.
3	NaN	Algeria	28.0339	1.6596	0	0	0	0	0	0	...	
4	NaN	Andorra	42.5063	1.5218	0	0	0	0	0	0	...	

5 rows × 498 columns

```
recovered_cases.columns
```

```
Index(['Column1', 'Column2', 'Column3', 'Column4', 'Column5', 'Column6',
       'Column7', 'Column8', 'Column9', 'Column10',
       ...
       'Column489', 'Column490', 'Column491', 'Column492', 'Column493',
       'Column494', 'Column495', 'Column496', 'Column497', 'Column498'],
      dtype='object', length=498)
```

```
recovered_cases.iloc[0]
```

```
Column1      Province/State
Column2      Country/Region
Column3                 Lat
Column4                Long
Column5             1/22/20
                       ...
Column494           5/25/21
Column495           5/26/21
Column496           5/27/21
Column497           5/28/21
Column498           5/29/21
Name: 0, Length: 498, dtype: object
```

```
recovered_cases.columns = recovered_cases.iloc[0]
recovered_cases = recovered_cases.drop(0).reset_index(drop = True)
recovered_cases
```

	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	...	5/20/
0	NaN	Afghanistan	33.93911	67.709953	0	0	0	0	0	0	...	556
1	NaN	Albania	41.1533	20.1683	0	0	0	0	0	0	...	1278
2	NaN	Algeria	28.0339	1.6596	0	0	0	0	0	0	...	879
3	NaN	Andorra	42.5063	1.5218	0	0	0	0	0	0	...	132
4	NaN	Angola	-11.2027	17.8739	0	0	0	0	0	0	...	264
...	...	...	...	...	...	...	...	...	...	...	...	
256	NaN	Vietnam	14.058324	108.277199	0	0	0	0	0	0	...	26
257	NaN	West Bank and Gaza	31.9522	35.2332	0	0	0	0	0	0	...	2956

```
# Melt -> Unpivot to the DataFrame
# pandas.melt
# pandas.melt(frame, id_vars=None, value_vars=None, var_name=None, value_name='value')
all_cols = confirmed_cases.columns
all_cols
```

```
Index(['Province/State', 'Country/Region', 'Lat', 'Long', '1/22/20', '1/23/20',
       '1/24/20', '1/25/20', '1/26/20', '1/27/20',
       ...
       '5/20/21', '5/21/21', '5/22/21', '5/23/21', '5/24/21', '5/25/21',
       '5/26/21', '5/27/21', '5/28/21', '5/29/21'],
      dtype='object', length=498)
```

```
id_vars = ['Province/State', 'Country/Region', 'Lat', 'Long']
date_cols = [c for c in all_cols if c not in id_vars] # List Comprehension
date_cols
```

```
['1/22/20',
 '1/23/20',
 '1/24/20',
 '1/25/20',
 '1/26/20',
 '1/27/20',
 '1/28/20',
 '1/29/20',
 '1/30/20',
 '1/31/20',
 '2/1/20',
 '2/2/20',
 '2/3/20',
 '2/4/20',
 '2/5/20',
 '2/6/20',
 '2/7/20',
```

```
date_cols = []
for c in all_cols:
    if c not in id_vars:
        date_cols.append(c)
date_cols
```

```
['1/22/20',
 '1/23/20',
 '1/24/20',
 '1/25/20',
 '1/26/20',
 '1/27/20',
 '1/28/20',
 '1/29/20',
 '1/30/20',
 '1/31/20',
 '2/1/20',
 '2/2/20',
 '2/3/20',
 '2/4/20',
 '2/5/20',
 '2/6/20',
 '2/7/20',
```

```
id_vars = confirmed_cases.iloc[:,:4].columns
id_vars
```

```
Index(['Province/State', 'Country/Region', 'Lat', 'Long'], dtype='object')
```

```
date_cols = confirmed_cases.iloc[:,4:].columns
date_cols
```

```
Index(['1/22/20', '1/23/20', '1/24/20', '1/25/20', '1/26/20', '1/27/20',
       '1/28/20', '1/29/20', '1/30/20', '1/31/20',
       ...
       '5/20/21', '5/21/21', '5/22/21', '5/23/21', '5/24/21', '5/25/21',
       '5/26/21', '5/27/21', '5/28/21', '5/29/21'],
      dtype='object', length=494)
```

```
confirmed_long = confirmed_cases.melt(
    id_vars = id_vars,
    value_vars = date_cols,
    var_name = "Date",
    value_name='Confirmed'
)
confirmed_long
```

	Province/State	Country/Region	Lat	Long	Date	Confirmed
0	NaN	Afghanistan	33.939110	67.709953	1/22/20	0
1	NaN	Albania	41.153300	20.168300	1/22/20	0
2	NaN	Algeria	28.033900	1.659600	1/22/20	0
3	NaN	Andorra	42.506300	1.521800	1/22/20	0
4	NaN	Angola	-11.202700	17.873900	1/22/20	0
...	...	...	...	...	...	...
136339	NaN	Vietnam	14.058324	108.277199	5/29/21	6908
136340	NaN	West Bank and Gaza	31.952200	35.233200	5/29/21	307838
136341	NaN	Yemen	15.552727	48.516388	5/29/21	6731
136342	NaN	Zambia	-13.133897	27.849332	5/29/21	94751
136343	NaN	Zimbabwe	-19.015438	29.154857	5/29/21	38933

136344 rows × 6 columns

```
deaths_long = deaths.melt(
    id_vars = id_vars,
    value_vars = date_cols,
    var_name = "Date",
    value_name='Deaths'
)
deaths_long
```

	Province/State	Country/Region	Lat	Long	Date	Deaths
0	NaN	Afghanistan	33.939110	67.709953	1/22/20	0.0
1	NaN	Albania	41.153300	20.168300	1/22/20	0.0
2	NaN	Algeria	28.033900	1.659600	1/22/20	0.0
3	NaN	Andorra	42.506300	1.521800	1/22/20	0.0
4	NaN	Angola	-11.202700	17.873900	1/22/20	0.0
...	...	...	...	...	...	...
136339	NaN	Vietnam	14.058324	108.277199	5/29/21	47.0
136340	NaN	West Bank and Gaza	31.952200	35.233200	5/29/21	3492.0
136341	NaN	Yemen	15.552727	48.516388	5/29/21	1319.0
136342	NaN	Zambia	-13.133897	27.849332	5/29/21	1276.0
136343	NaN	Zimbabwe	-19.015438	29.154857	5/29/21	1594.0

136344 rows × 6 columns

```python
recovered_long = recovered_cases.melt(
    id_vars = id_vars,
    value_vars = date_cols,
    var_name = "Date",
    value_name='Recovered'
)
recovered_long
```

	Province/State	Country/Region	Lat	Long	Date	Recovered
0	NaN	Afghanistan	33.93911	67.709953	1/22/20	0
1	NaN	Albania	41.1533	20.1683	1/22/20	0
2	NaN	Algeria	28.0339	1.6596	1/22/20	0
3	NaN	Andorra	42.5063	1.5218	1/22/20	0
4	NaN	Angola	-11.2027	17.8739	1/22/20	0
...	...	...	...	...	...	...
128929	NaN	Vietnam	14.058324	108.277199	5/29/21	2896
128930	NaN	West Bank and Gaza	31.9522	35.2332	5/29/21	300524
128931	NaN	Yemen	15.552727	48.516388	5/29/21	3399
128932	NaN	Zambia	-13.133897	27.849332	5/29/21	91594
128933	NaN	Zimbabwe	-19.015438	29.154857	5/29/21	36578

128934 rows × 6 columns

```python
# Structural Insights for Each DataFrame.....
confirmed_long.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 136344 entries, 0 to 136343
Data columns (total 6 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   Province/State  42484 non-null   object
 1   Country/Region  136344 non-null  object
 2   Lat             135356 non-null  float64
 3   Long            135356 non-null  float64
 4   Date            136344 non-null  object
 5   Confirmed       136344 non-null  int64
dtypes: float64(2), int64(1), object(3)
memory usage: 6.2+ MB
```

```python
# Identifying the missing values
confirmed_long['Province/State'].isna().sum()
```

```
93860
```

```python
confirmed_long['Province/State'].value_counts(dropna = False)
```

```
Province/State
NaN                        93860
Unknown                      494
Guadeloupe                   494
French Polynesia             494
French Guiana                494
                            ...
Chongqing                    494
Beijing                      494
Anhui                        494
Yukon                        494
Turks and Caicos Islands     494
Name: count, Length: 87, dtype: int64
```

```python
# Confirmed_Long -> [Data Cleaning and Handling Missing Values]
confirmed_long['Province/State'] = confirmed_long['Province/State'].fillna('All Provinces')
confirmed_long['Lat'] = pd.to_numeric(confirmed_long['Lat'].fillna(0))
confirmed_long['Long'] = pd.to_numeric(confirmed_long['Long'].fillna(0))
confirmed_long['Date'] = pd.to_datetime(confirmed_long['Date'] , format = "%m/%d/%y" , errors = 'coerce')
confirmed_long.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 136344 entries, 0 to 136343
Data columns (total 6 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   Province/State  136344 non-null  object
 1   Country/Region  136344 non-null  object
 2   Lat             136344 non-null  float64
 3   Long            136344 non-null  float64
 4   Date            136344 non-null  datetime64[ns]
 5   Confirmed       136344 non-null  int64
dtypes: datetime64[ns](1), float64(2), int64(1), object(2)
memory usage: 6.2+ MB
```

```python
confirmed_long.head()
```

	Province/State	Country/Region	Lat	Long	Date	Confirmed
0	All Provinces	Afghanistan	33.93911	67.709953	2020-01-22	0
1	All Provinces	Albania	41.15330	20.168300	2020-01-22	0
2	All Provinces	Algeria	28.03390	1.659600	2020-01-22	0
3	All Provinces	Andorra	42.50630	1.521800	2020-01-22	0
4	All Provinces	Angola	-11.20270	17.873900	2020-01-22	0

```python
deaths_long.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 136344 entries, 0 to 136343
Data columns (total 6 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   Province/State  42484 non-null   object
 1   Country/Region  136344 non-null  object
 2   Lat             135356 non-null  float64
 3   Long            135356 non-null  float64
 4   Date            136344 non-null  object
 5   Deaths          136343 non-null  float64
dtypes: float64(3), object(3)
memory usage: 6.2+ MB
```

```python
# Deaths_long-> [Data Cleaning and Handling Missing Values]
deaths_long['Province/State'] = deaths_long['Province/State'].fillna('All Provinces')
deaths_long['Lat'] = pd.to_numeric(deaths_long['Lat'].fillna(0))
deaths_long['Long'] = pd.to_numeric(deaths_long['Long'].fillna(0))
deaths_long['Date'] = pd.to_datetime(deaths_long['Date'] , format = "%m/%d/%y" , errors = 'coerce')
deaths_long['Deaths'] = pd.to_numeric(deaths_long['Deaths'].fillna(0).astype('int64'))
deaths_long.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 136344 entries, 0 to 136343
Data columns (total 6 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   Province/State  136344 non-null  object
 1   Country/Region  136344 non-null  object
 2   Lat             136344 non-null  float64
 3   Long            136344 non-null  float64
 4   Date            136344 non-null  datetime64[ns]
 5   Deaths          136344 non-null  int64
dtypes: datetime64[ns](1), float64(2), int64(1), object(2)
memory usage: 6.2+ MB
```

```python
deaths_long.head()
```

	Province/State	Country/Region	Lat	Long	Date	Deaths
0	All Provinces	Afghanistan	33.93911	67.709953	2020-01-22	0
1	All Provinces	Albania	41.15330	20.168300	2020-01-22	0
2	All Provinces	Algeria	28.03390	1.659600	2020-01-22	0
3	All Provinces	Andorra	42.50630	1.521800	2020-01-22	0
4	All Provinces	Angola	-11.20270	17.873900	2020-01-22	0

```python
recovered_long.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 128934 entries, 0 to 128933
Data columns (total 6 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   Province/State  34580 non-null   object
 1   Country/Region  128934 non-null  object
 2   Lat             128440 non-null  object
 3   Long            128440 non-null  object
 4   Date            128934 non-null  object
 5   Recovered       128933 non-null  object
dtypes: object(6)
memory usage: 5.9+ MB
```

```python
# Recovered_Long -> [Data Cleaning and Handling Missing Values]
recovered_long['Province/State'] = recovered_long['Province/State'].fillna('All Provinces')
recovered_long['Lat'] = pd.to_numeric(recovered_long['Lat'].fillna(0))
recovered_long['Long'] = pd.to_numeric(recovered_long['Long'].fillna(0))
recovered_long['Date'] = pd.to_datetime(recovered_long['Date'] , format = "%m/%d/%y" , errors = 'coerce')
recovered_long['Recovered'] = pd.to_numeric(recovered_long['Recovered'].fillna(0).astype('int64'))
recovered_long.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 128934 entries, 0 to 128933
Data columns (total 6 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   Province/State  128934 non-null  object
 1   Country/Region  128934 non-null  object
 2   Lat             128934 non-null  float64
 3   Long            128934 non-null  float64
 4   Date            128934 non-null  datetime64[ns]
 5   Recovered       128934 non-null  int64
dtypes: datetime64[ns](1), float64(2), int64(1), object(2)
memory usage: 5.9+ MB
```

```python
recovered_long.head()
```

	Province/State	Country/Region	Lat	Long	Date	Recovered
0	All Provinces	Afghanistan	33.93911	67.709953	2020-01-22	0
1	All Provinces	Albania	41.15330	20.168300	2020-01-22	0
2	All Provinces	Algeria	28.03390	1.659600	2020-01-22	0
3	All Provinces	Andorra	42.50630	1.521800	2020-01-22	0
4	All Provinces	Angola	-11.20270	17.873900	2020-01-22	0

```
type(confirmed_long)
```

```
pandas.core.frame.DataFrame
```

```
print(type(confirmed_long))
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
# GroupBY + Aggregations [Feature Selection]
# cd stands for [Country + Date]
confirmed_cd = (
    confirmed_long.groupby(['Country/Region' , 'Date'])['Confirmed'].sum().reset_index()
)
confirmed_cd
```

	Country/Region	Date	Confirmed
0	Afghanistan	2020-01-22	0
1	Afghanistan	2020-01-23	0
2	Afghanistan	2020-01-24	0
3	Afghanistan	2020-01-25	0
4	Afghanistan	2020-01-26	0
...	...	...	...
95337	Zimbabwe	2021-05-25	38706
95338	Zimbabwe	2021-05-26	38819
95339	Zimbabwe	2021-05-27	38854
95340	Zimbabwe	2021-05-28	38918
95341	Zimbabwe	2021-05-29	38933

```
# GroupBY + Aggregations [Feature Selection]
# cd stands for [Country + Date]
recovered_cd = (
    recovered_long.groupby(['Country/Region' , 'Date'])['Recovered'].sum().reset_index()
)
recovered_cd
```

	Country/Region	Date	Recovered
0	Afghanistan	2020-01-22	0
1	Afghanistan	2020-01-23	0
2	Afghanistan	2020-01-24	0
3	Afghanistan	2020-01-25	0
4	Afghanistan	2020-01-26	0
...	...	...	...
95337	Zimbabwe	2021-05-25	36517
95338	Zimbabwe	2021-05-26	36531
95339	Zimbabwe	2021-05-27	36541
95340	Zimbabwe	2021-05-28	36563
95341	Zimbabwe	2021-05-29	36578

95342 rows × 3 columns

```python
# pandas.merge
# pandas.merge(left, right, how='inner', on=None)
merged = (confirmed_cd
          .merge(deaths_cd , on = ['Country/Region','Date'], how = 'inner')
          .merge(recovered_cd , on = ['Country/Region','Date'], how = 'inner')
          .sort_values(['Country/Region','Date'])
)
merged
```

	Country/Region	Date	Confirmed	Deaths	Recovered
0	Afghanistan	2020-01-22	0	0	0
1	Afghanistan	2020-01-23	0	0	0
2	Afghanistan	2020-01-24	0	0	0
3	Afghanistan	2020-01-25	0	0	0
4	Afghanistan	2020-01-26	0	0	0
...	...	...	...	...	...
95337	Zimbabwe	2021-05-25	38706	1587	36517
95338	Zimbabwe	2021-05-26	38819	1589	36531
95339	Zimbabwe	2021-05-27	38854	1592	36541
95340	Zimbabwe	2021-05-28	38918	1592	36563
95341	Zimbabwe	2021-05-29	38933	1594	36578

95342 rows × 5 columns

```python
merged.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 95342 entries, 0 to 95341
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Country/Region  95342 non-null  object
 1   Date            95342 non-null  datetime64[ns]
 2   Confirmed       95342 non-null  int64
 3   Deaths          95342 non-null  int64
 4   Recovered       95342 non-null  int64
dtypes: datetime64[ns](1), int64(3), object(1)
memory usage: 3.6+ MB
```

```python
# 2.2 Generate Plots of Confirmed Cases over time for Top Countries.....
import pandas as pd
import matplotlib.pyplot as plt

latest_dates = merged.groupby('Country/Region')['Date'].max()
latest_dates
```

```
Country/Region
Afghanistan          2021-05-29
Albania              2021-05-29
Algeria              2021-05-29
Andorra              2021-05-29
Angola               2021-05-29
                        ...
Vietnam              2021-05-29
West Bank and Gaza   2021-05-29
Yemen                2021-05-29
Zambia               2021-05-29
Zimbabwe             2021-05-29
Name: Date, Length: 193, dtype: datetime64[ns]
```

```python
# New DataFrame
latest_totals = merged.copy() # Shallow Copy
latest_totals = latest_totals[latest_totals['Date'] == '2021-05-29']
latest_totals
```

	Country/Region	Date	Confirmed	Deaths	Recovered
493	Afghanistan	2021-05-29	70111	2899	57281
987	Albania	2021-05-29	132297	2449	129215
1481	Algeria	2021-05-29	128456	3460	89419
1975	Andorra	2021-05-29	13693	127	13416
2469	Angola	2021-05-29	34180	757	27646
...	...	...	...	...	...
93365	Vietnam	2021-05-29	6908	47	2896
93859	West Bank and Gaza	2021-05-29	307838	3492	300524
94353	Yemen	2021-05-29	6731	1319	3399
94847	Zambia	2021-05-29	94751	1276	91594
95341	Zimbabwe	2021-05-29	38933	1594	36578

193 rows × 5 columns

```python
latest_totals = latest_totals.set_index('Country/Region')['Confirmed']
latest_totals
```

```
Country/Region
Afghanistan            70111
Albania               132297
Algeria               128456
Andorra                13693
Angola                 34180
                        ...
Vietnam                 6908
West Bank and Gaza    307838
Yemen                   6731
Zambia                 94751
Zimbabwe               38933
Name: Confirmed, Length: 193, dtype: int64
```

```python
top_5_countries = latest_totals.nlargest(5)
top_5_countries
```

```
Country/Region
US        33251939
India     27894800
Brazil    16471600
France     5719877
Turkey     5235978
Name: Confirmed, dtype: int64
```